



Assessing RAG: Where it breaks—and can we fix it?

Completion requirements

Assessing RAG: Where it breaks—and can we fix it?

In this session, we reviewed the naive RAG pipeline, identified common failure points from curation through generation, and discussed practical ways to improve reliability and faithfulness.

Naive RAG Pipeline (Review)

- **Curate corpus:** select, clean, and organize source documents to create the reference material for retrieval.
- **Ingest and index:** split documents into **chunks**, embed them with a **neural embedding model**, and store those **vectors** in a database for fast lookup.
- **Query:** take the user's question (as-is or lightly normalized) and embed it with the **same model** used for the corpus.
- **Retrieve:** perform a **K-nearest neighbors (KNN)** search over the index to find the most relevant chunks.
- **Assemble context:** combine the retrieved chunks into a short **context** that fits within the **generation model's context window**.
- **Generate:** have the **language model (LLM)** answer using only that context, or **abstain** if there is not enough information.
- **Return:** present the **answer**, optionally with **citations** to the retrieved sources.

Common Problems with the Naive RAG Pipeline

- **Curation and ingestion issues**
 - Incomplete, outdated, or poorly organized sources yield weak retrieval.
 - **Redundant** or **inconsistent** documents confuse the model and slow retrieval.
 - *Garbage in, garbage out.*
- **Chunking problems**
 - Chunks too small lose context; too large dilute meaning or exceed **token limits**.
 - Poor **boundaries** (key sentences split across chunks) block retrieval of full ideas.
- **Embedding mismatches**

- The **embedding model** may not capture the domain or language of the corpus.
- Using different models for corpus and query breaks the shared **vector space**.
- Domain-specific embeddings often outperform general-purpose ones.
- **Retrieval errors**
 - **KNN** can return passages that are **semantically similar** but not actually relevant.
 - **Dense retrieval** struggles with exact names, numbers, or symbols.
 - Similarity search can overweight generic phrasing and miss precise facts.
- **Context assembly issues**
 - Naively concatenating top chunks can bury evidence or cause repetition.
 - Long contexts risk the **lost-in-the-middle** effect or exceed limits.
 - Ordering and light compression improve relevance and speed.
- **Generation failures**
 - The model may **hallucinate** content not supported by context.
 - It can blend unrelated details from multiple chunks.
 - Without strict instructions to “use only the provided context,” prior training can override retrieved evidence.
- **Faithfulness and grounding**
 - Even when quoting, subtle **distortions** can creep in.
 - True **faithfulness** means every claim is supported by retrieved text, not just related to it.
 - Evaluation often needs manual or rubric-based checking.
- **Latency and scaling bottlenecks**
 - Each stage—embedding, retrieval, generation—adds time.
 - Larger corpora slow **indexing** and **similarity search**.
 - Long contexts increase computation and cost.
- **Evaluation difficulties**
 - RAG quality spans **retrieval accuracy**, **faithfulness**, and **usefulness**.
 - Without clear **metrics**, it’s hard to gauge improvement.
 - Fixing one issue (e.g., recall) can worsen another (e.g., latency).

Improving the Naive RAG Pipeline

Parallel Keyword / BM25 Path

- Generate or extract **keywords** or short **key phrases** from the query.
- Run **BM25** alongside dense retrieval.
- Fuse results (e.g., **Reciprocal Rank Fusion, RRF**) before selecting top candidates.
- Hybrid retrieval recovers exact matches (IDs, names, phrases) that dense embeddings often miss.

HyDE (Hypothetical Document Embeddings)

- Ask an LLM to draft a short **hypothetical answer** to the query.
- Embed that draft, then retrieve **real** documents nearest to it in vector space.
- Bridges short queries to longer passages and typically boosts recall with a small latency trade-off.

Re-Ranking (Two-Stage Retrieval)

- After initial retrieval, apply a **re-ranker** (cross-encoder or small LLM) to rescore and reorder candidates.
- Improves **precision**, pushing the best evidence to the top.
- Common pattern: fast KNN retrieval → slower re-ranking on a narrowed candidate set.

De-Duplication and Versioning

- **De-duplication**: remove or merge near-identical chunks to reduce redundancy and conflicts.
- **Version tracking**: tag with dates or versions so retrieval prefers current, authoritative sources.

Hybrid and Diversity-Aware Retrieval

- Combine **dense** (semantic) and **sparse** (BM25) methods for coverage and precision.
- Encourage **diversity** among top results to cover multiple perspectives, useful for multi-hop tasks.

Approximate Nearest Neighbors (ANN) Search

- Find top-K neighbors **without exhaustive comparisons**, trading slight accuracy for big speed gains.
- Common implementations: **FAISS**, **ScaNN**, **HNSW**.
- Enables real-time retrieval over thousands to millions of chunks.

Feedback and Corrective Loops

- Add a **self-critique** step: if confidence or grounding is low, reformulate the query and search again.
- Allow **abstention** or clarification requests instead of forcing weak answers.
- These loops are early steps toward **agent-like** adaptive RAG.

Metrics and Evaluation

- **Hit@K** – Fraction of queries where at least one correct item appears in the top K.
- **Recall@K** – (Number of relevant items in top K) ÷ (Total relevant items that exist for the query).

- **MRR (Mean Reciprocal Rank)** – Average of $1/\text{rank}$ of the first correct result.
- **Faithfulness (Groundedness)** – Are claims supported directly by retrieved text?
- **Accuracy / Exact Match (EM)** – For simple QA, whether the final answer exactly matches the reference.
- **Latency** – Time to respond. p50 = median; p95/p99 capture slow tails.
- **Throughput** – Queries handled per second (capacity).
- **Cost** – Compute time, tokens, or dollars for embedding/inference.
- **nDCG (Normalized Discounted Cumulative Gain)** – IR metric that rewards relevant items near the top. (Mentioned for awareness; not covered this year.)

A Hypothetical Improved RAG Pipeline

- **Curate corpus:** clean, de-duplicate, and remove outdated versions before indexing.
- **Ingest and index:** chunk sensibly; embed with a neural model; store vectors in an index.
- **Query:** embed the question with the same model; also extract keywords for **parallel BM25**.
- **Retrieve:**
 - **Dense retrieval (ANN):** get top-25 chunks.
 - **BM25:** get top-25 matches.
 - **Fuse:** combine lists (e.g., **RRF**) into a top-50 candidate set.
- **Re-rank:** apply a cross-encoder or small LLM to select the final 5–8 chunks.
- **Assemble context:** order for salience (front-load crucial evidence) to mitigate **lost-in-the-middle**.
- **Generate:** instruct the LLM to answer **strictly from context** or abstain if evidence is insufficient.
- **Evaluate response:** a second **LLM call** checks **faithfulness** and **citation quality**.
- **Return:** present answer + citations; log retrieval and response metrics for analysis.

Assessing the Pipeline

1. **Retrieval performance:** Hit@K, Recall@K, MRR (relevant chunks retrieved early and consistently).
2. **Answer quality:** faithfulness, citation accuracy, and EM/accuracy where applicable.
3. **System performance:** latency (per stage and end-to-end), throughput, cost—watch for bottlenecks.

Overall success balances **precision, recall, faithfulness, and speed** to deliver accurate, well-cited responses efficiently.

Recommended Videos

Understanding BM25 and Hybrid Retrieval (YouTube) (~9 min.)



Play Video

2 Methods For Improving Retrieval in RAG

This video illustrates how **keyword-based methods like BM25** complement **dense neural retrieval** in RAG systems.

How to use Retrieval Augmented Generation (Google Cloud Tech) (~6 min.)

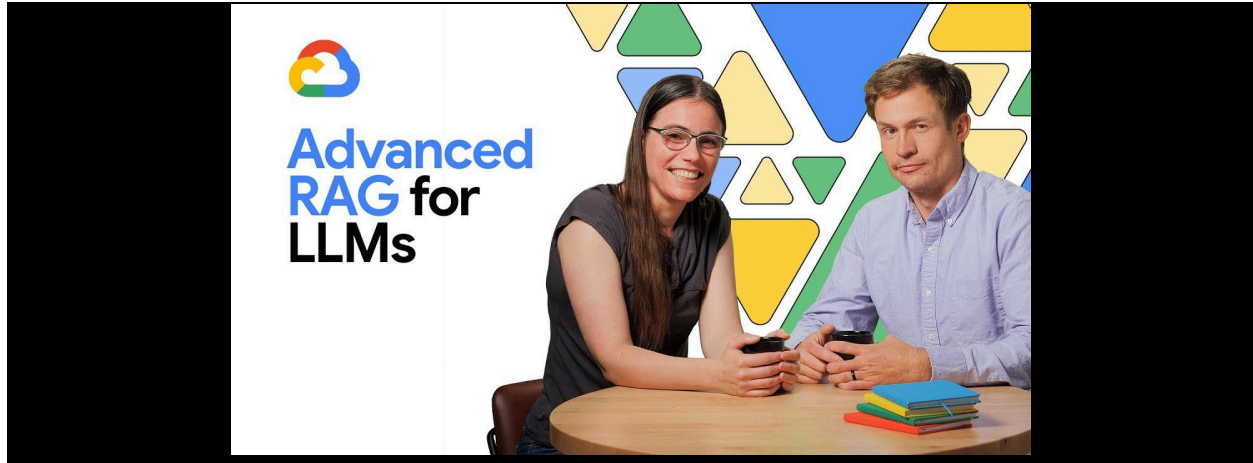


Play Video

How to use Retrieval Augmented Generation (RAG)

A concise Google Cloud Tech walkthrough of the **basic RAG workflow**, showing how retrieval and generation combine to produce grounded responses.

Advanced RAG Techniques for Developers (Google Cloud Tech) (~9 min.)



Play Video

▶ Advanced RAG techniques for developers

A follow-up from Google Cloud Tech covering **context expansion**, **hybrid retrieval**, and **re-ranking** in production systems. (They don't use the term **HyDE**, but they touch on the underlying idea.)

Glossary of Key Terms

Abstain: The model declines to answer due to insufficient evidence.

Approximate Nearest Neighbors (ANN): Fast top-K similarity without exhaustive comparisons (e.g., **FAISS**, **ScaNN**, **HNSW**). *Not* an artificial neural network.

BM25: Keyword-based ranking that favors term frequency and rarity; great for exact matches (IDs, names, numbers).

Chunk / Chunking: Splitting documents into pieces for embedding/retrieval; size and overlap control context retention.

Context window: Max tokens an LLM can attend to in one pass; exceeding it truncates information.

Corpus (corpora): The collection of source documents used for retrieval.

Cross-encoder / Re-ranker: Model that scores query–document pairs to re-order candidates with higher precision.

De-duplication: Removing or merging near-identical content to reduce redundancy/conflict.

Dense retrieval: Retrieval via **neural embeddings** (semantic similarity) rather than raw keywords.

Embedding / Embedding model: Vector representation of text; **use the same model** for corpus and queries to share a vector space.

Faithfulness (Groundedness): Every claim in the answer is supported by retrieved evidence.

Fusion / RRF: Merging multiple ranked lists (BM25 + dense) into a single ranking.

Hallucination: Plausible-sounding but unsupported or false content from an LLM.

Hit@K: Fraction of queries where at least one correct item appears in the top K.

Hybrid retrieval: Combining dense and sparse (BM25) retrieval for coverage and precision.

HyDE: LLM-generated hypothetical answer used to retrieve real documents nearby in vector space.

Index: The (vector) database holding embedded chunks.

KNN: K-nearest neighbors search returning the K most similar items.

Latency: The time it takes for a system to return results. p50 = median; p95 and p99 measure slower outliers.

Lost-in-the-middle problem: Models underweight information placed mid-prompt in long contexts.

MRR (Mean Reciprocal Rank): A ranking metric that averages $1/\text{rank}$ of the first correct result across many queries.

nDCG (Normalized Discounted Cumulative Gain): IR metric that gives more weight to relevant items near the top of the list.

Precision / Recall: Precision = fraction of retrieved items that are relevant; Recall = fraction of relevant items that were retrieved.

Re-ranking: Second-stage ordering to improve precision, often with a cross-encoder or small LLM.

RRF: (See **Fusion / RRF**.)

Throughput: Queries processed per second (system capacity).

Token: Internal unit of text (word/subword/character) for LLMs.

Vector space: The high-dimensional space where embeddings are compared.

Version tracking: Metadata (date/version) so retrieval prefers current/authoritative sources.

Last modified: Tuesday, October 14, 2025, 8:21 PM